Every Ware Lab

UNIVERSITÀ DEGLI STUDI DI MILANO

Graph Neural Networks for Ambient Intelligence

Ambient Intelligence and Domotics

Master Degree: Artificial Intelligence for Science and Technology

Gabriele Civitarese

EveryWare Lab, Department of Computer Science

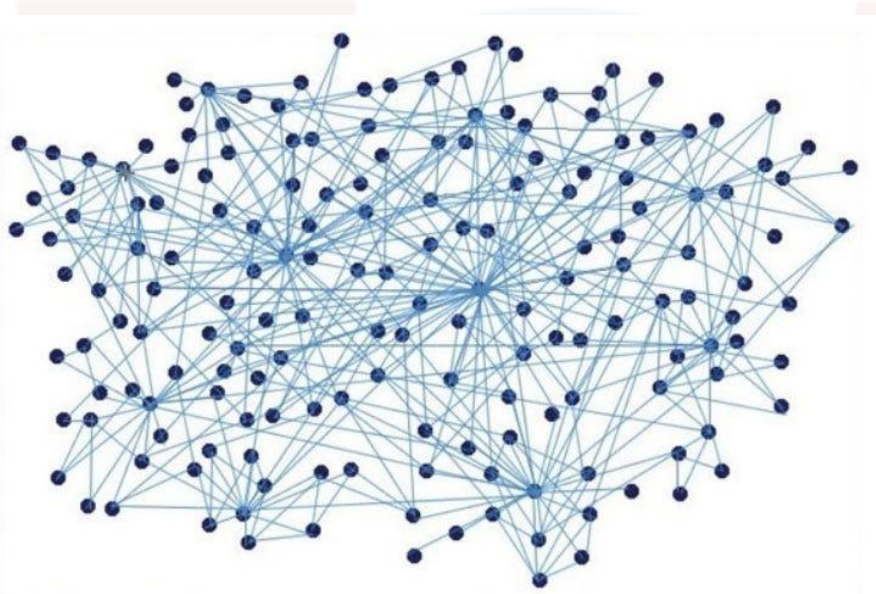
Università degli Studi di Milano

Copyright

Every Ware Lab

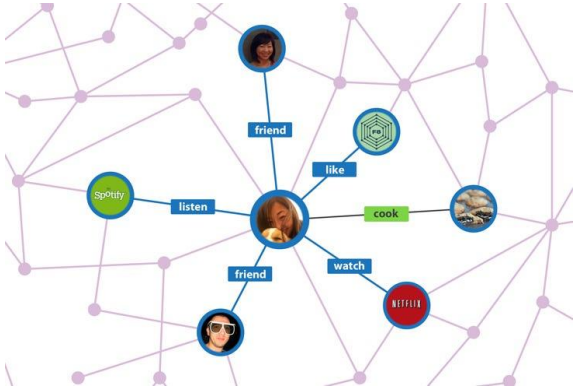
What is a graph?

- A graph is a collection of objects *(nodes)* and their interactions *(edges)*
  - e.g., in a Social Network each person is a node, and an edge between two persons indicate that they are friends
- Why the graph formalism is powerful?
  - to leverage relationships between data points
  - it can be applied to many different domains
  - graphs offer math foundations to analyze, understand and learn complex systems

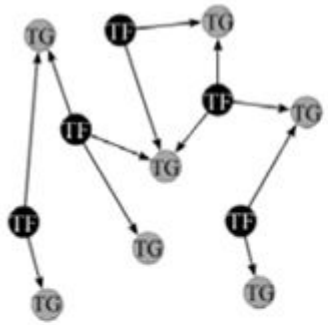


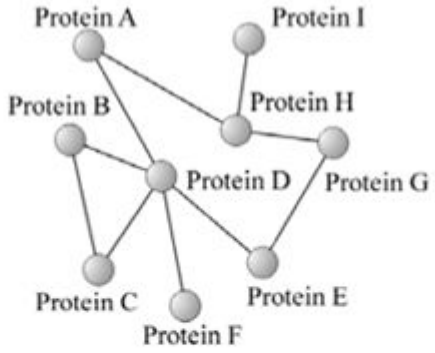
Every Ware Lab

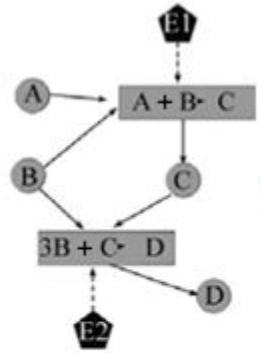
Graphs are everywhere



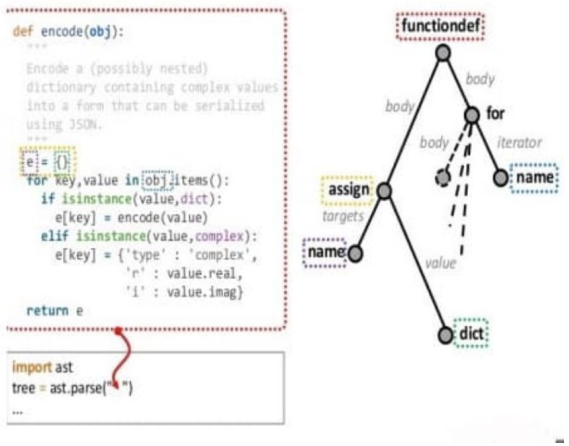
Social networks



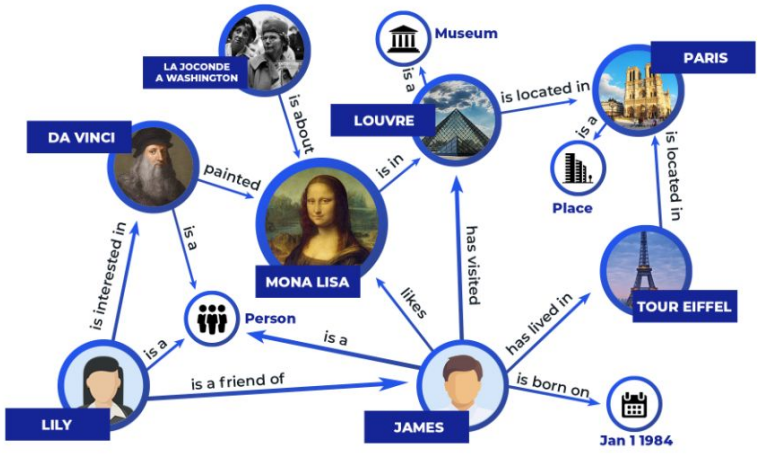




Biology



Software

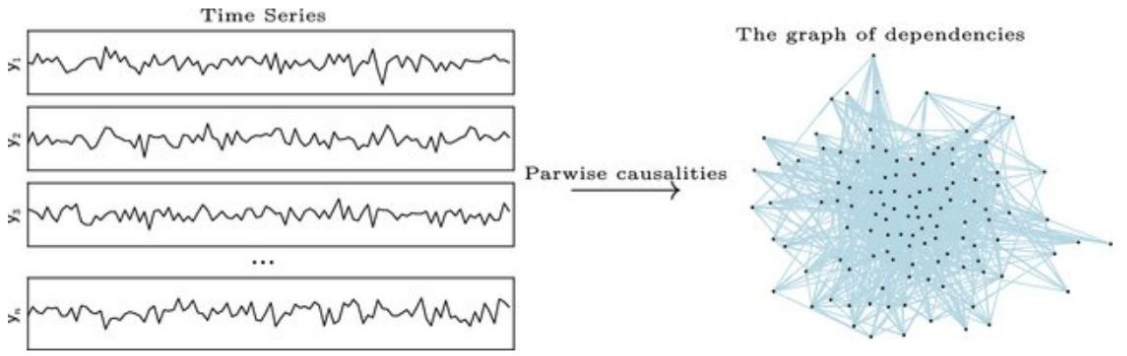


Knowledge Graphs

Every Ware Lab

Graphs and Time Series?

- Recently, several research groups proposed ML models to process time series encoded as graphs
- Why?
  - they can capture connections between variables of a multivariate time series
  - they can capture dependencies between different points in time
- This lecture will include the following:
  - basic machine learning tasks with graphs
  - introduction to GNNs
  - how to apply GNNs to time-series
  - applications in Ambient Intelligence

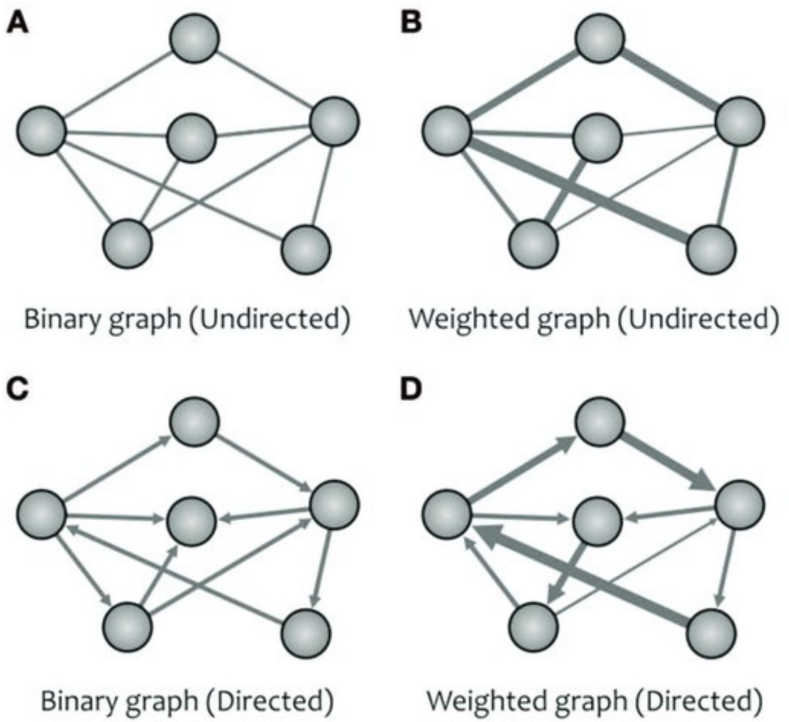


Every Ware Lab

Preliminaries on graphs

Graph definition

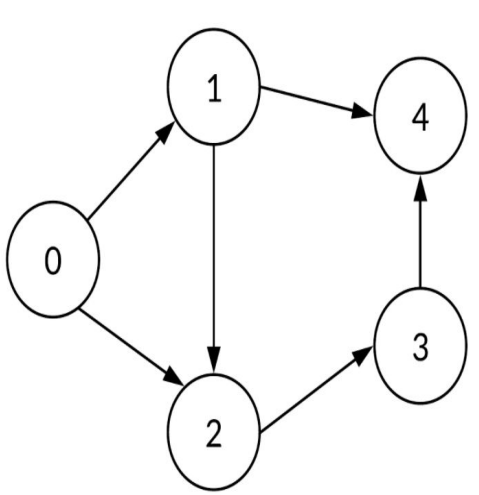
- A graph $G = (\boldsymbol{V,E})$ is represented by the set of nodes **V** and set of edges **E**
- If there is an edge from a node $u \in \mathbf{V}$ to another node $v \in \mathbf{V}$, then $(u,v) \in \mathbf{E}$
- Types of graphs:
  - ***undirected***
    - if $(u,v) \in \mathbf{E}$ then $(v,u) \in \mathbf{E}$
  - ***directed***
    - an edge $(u,v) \in \mathbf{E}$ goes from node u to node v
  - ***weighted***
    - each edge is associated with a weight
    - both undirected and directed graphs may be weighted



Every Ware Lab

Adjacency Matrix

- Graphs are represented by the Adjacency Matrix $A \in \mathfrak{R}^{|V|*|V|}$
  - A[u,v] = 1 if (u,v)$\in$**E**
  - 0 otherwise
- There is an explicit ordering so that every node identifies a particular row and column
- If the graph is undirected, the matrix is symmetric
- If the graph is directed (i.e., the edge direction matters), the matrix may be not symmetric



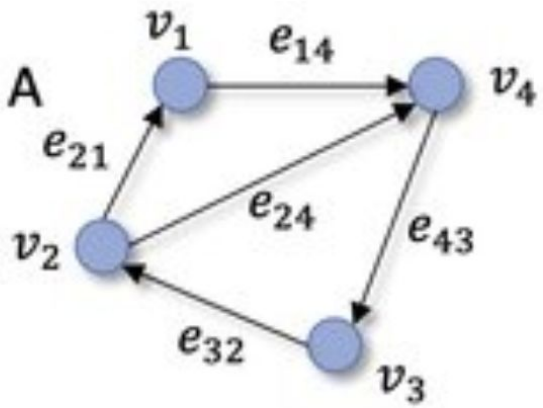
Adjacency Matrix

	0	1	2	3	4
0	0	1	1	0	0
1	0	0	1	0	1
2	0	0	0	1	0
3	0	0	0	0	1
4	0	0	0	0	0

Every Ware Lab

Examples of Adjacency Matrices

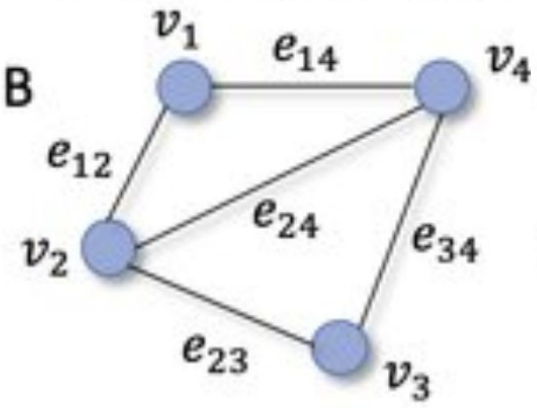
Directed graph G(V,E)



E

	v_1	v_2	v_3	v_4
v_1	0	0	0	1
v_2	1	0	0	1
v_3	0	1	0	0
v_4	0	0	1	0

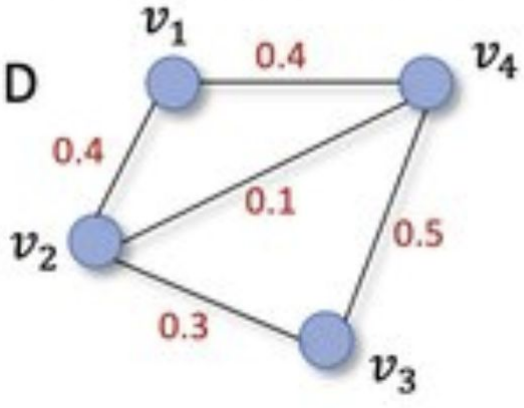
Undirected graph G(V,E)



F

	v_1	v_2	v_3	v_4
v_1	0	1	0	1
v_2	1	0	1	1
v_3	0	1	0	1
v_4	1	1	1	0

Weighted graph G(V,E)



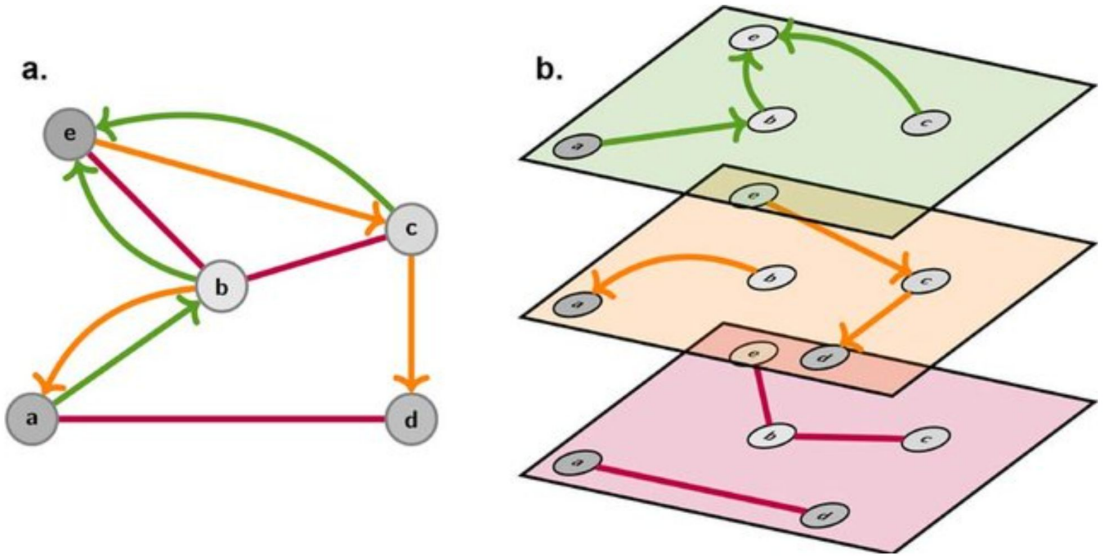
H

	v_1	v_2	v_3	v_4
v_1	0	0.4	0	0.4
v_2	0.4	0	0.3	0.1
v_3	0	0.3	0	0.5
v_4	0.4	0.1	0.5	0

Every Ware Lab

Multi-relational graphs

- Extension of graphs where each relation also has a type
- $(u,\tau,v) \in \mathbf{E} \rightarrow$ nodes *u* and *v* are connected by a relation of type τ
- There is one adjacency A_τ matrix for each relation τ
- Overall adjacency matrix A
  - $A \in \mathfrak{R}^{|V|*|R|*|V|}$
    - where R is the set of all the possible relations



Heterogeneous Graphs

- Multi-relational graphs where also the nodes are typed
- The set of nodes **V** is partitioned in subsets
- $\mathbf{V} = V_1 \cup V_2 \cup \ldots \cup V_k$
  - $\forall\ i \neq j\ V_i \cap V_j = \varnothing$
- Each partition V_i only includes nodes of the same type
- Defining edges also implies having constraints on node types
  - e.g., $(u,\boldsymbol{\tau},v) \in \mathbf{E}$ implies that $u \in V_j$ and $v \in V_k$
- Example:
  - node types: proteins, drugs, and diseases
  - the relationship “treatment” is defined only between drugs and disease nodes

Every Ware Lab

Node-level Attributes

- In a graph, each node is usually associated with a set of features
  - These are referred as ***node-level attributes***
- The overall node-level attributes of a graph can be represented as
  - $\mathbf{X} \in \mathfrak{R}^{|V|*m}$
    - where *m* is the dimensionality of the node-level attributes
    - assumption: the ordering of nodes is consistent with the ones in the adjacency matrix
- In heterogeneous graphs, each node type has its own distinct attributes!

Every Ware Lab

Machine Learning on Graphs

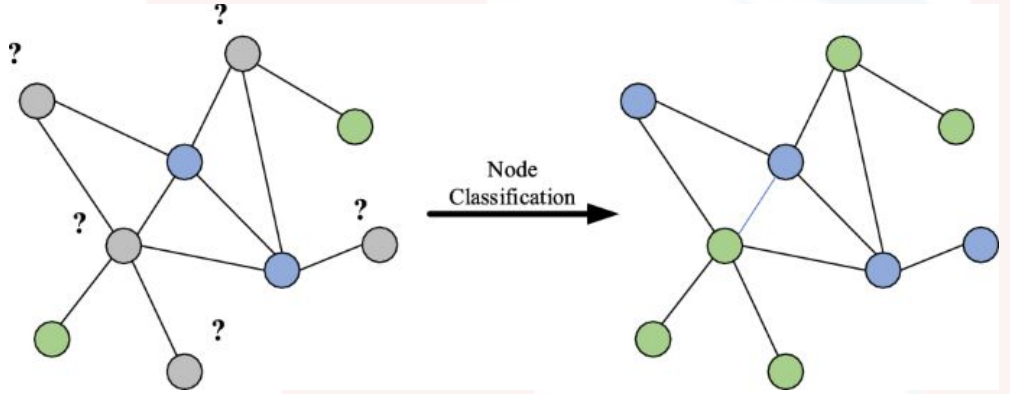
Machine Learning tasks for Graphs

- The most popular machine learning tasks on graphs are:
  - Node Classification
  - Link Prediction
  - Graph classification/regression/clustering
- In the following, these tasks will be described

Every Ware Lab

Node Classification

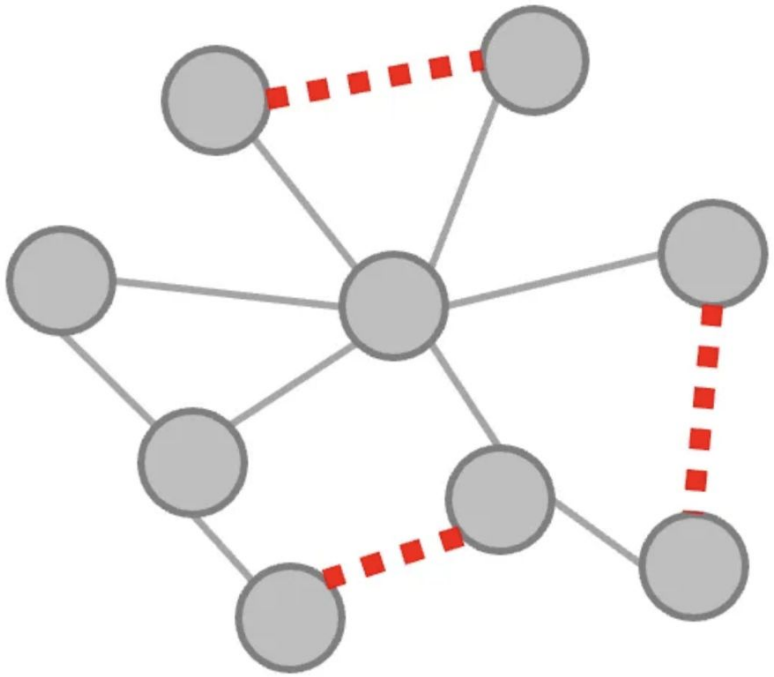
- **Goal:** predict the label y_u for each $u \in \mathbf{V}$ given that labels are available for $V_{train} \subset \mathbf{V}$
- Two common scenarios:
  - ***semi-supervised learning:*** there are a few labeled nodes in the graph and the goal is to classify the unlabeled ones
  - ***supervised learning:*** there are many labeled nodes and the goal is to generalize across disconnected graphs
- Challenge: nodes may be non-IID and connections between them should be leveraged
- Examples of applications:
  - identify bots in social networks, classify documents topic based on hyperlinks



Every Ware Lab

Link Prediction

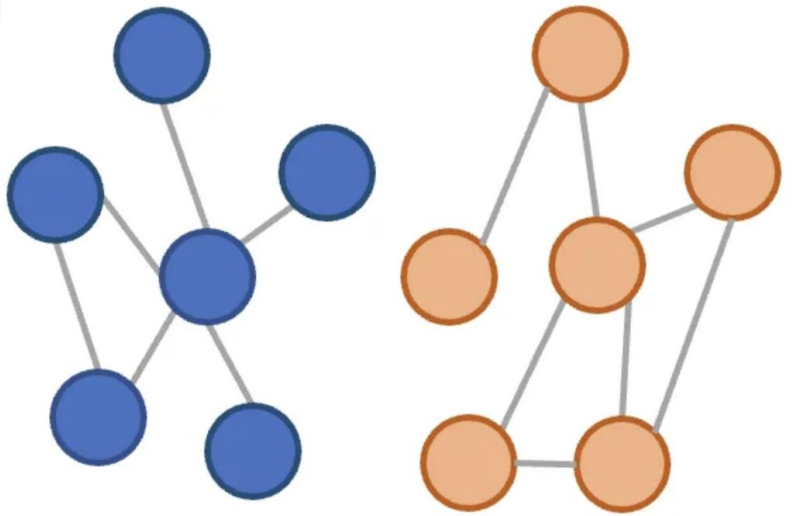
- Given a set of nodes **V** and an *incomplete* set of edges $E_{train} \subset \mathbf{E}$, the goal is to infer the missing edges
- Example of application:
  - suppose that we have a graph of interactions between proteins, but we only know only some of them. The goal is to infer unknown interactions



Every Ware Lab

Graph classification/regression

- Until now, the tasks involved predictions on individual components (i.e., edges, nodes)
- Given a dataset of multiple different graphs, it is also possible to make independent predictions on each graph (classification or regression)
- Similarly to standard supervised learning, each graph is a data point associated with a label, and the goal is mapping data points to labels
- Example of application:
  - given a graph-based representation of syntax and data flow, determine if a computer program is malicious



Every Ware Lab

Graph Clustering

- The unsupervised counterpart of Graph Classification
- Goal: learning an unsupervised measure of similarity between graphs
- Again, it is the “straightforward” extension of clustering on graph data

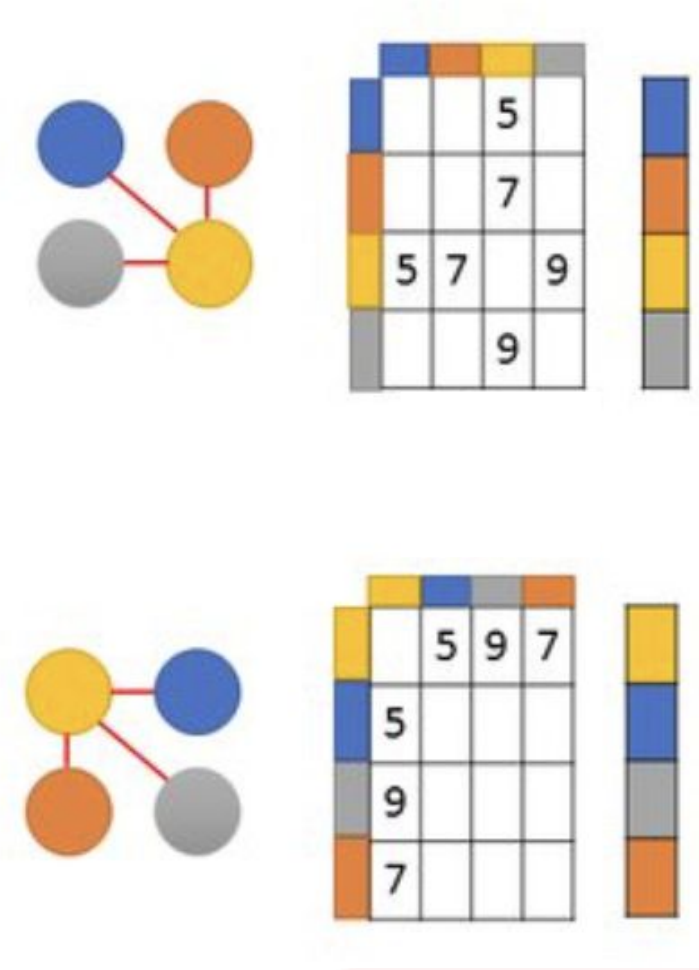
Challenges

- While these last two tasks seem to be very similar to standard supervised or unsupervised solutions…
- …the challenge is to find features taking into account the graph relational structure!
- A naive approach:
  - just use the adjacency matrix as input (e.g., flattened and provided to an MLP model)
  - **why doesn't it work?**
    - the adjacency matrix has an arbitrary ordering of nodes
    - this representation is not permutation-invariant

Every Ware Lab

Graph Isomorphism

- These two graphs represent exactly the same thing, even though they reflect two different ***permutations*** of their nodes
  - i.e., they are *isomorphic* to each other
- However, their adjacency matrices are completely different
- The goal of GNNs is to determine a permutation independent graph representation!
  - ***idea:*** the nodes should be considered considered as a *set* where the order is not relevant

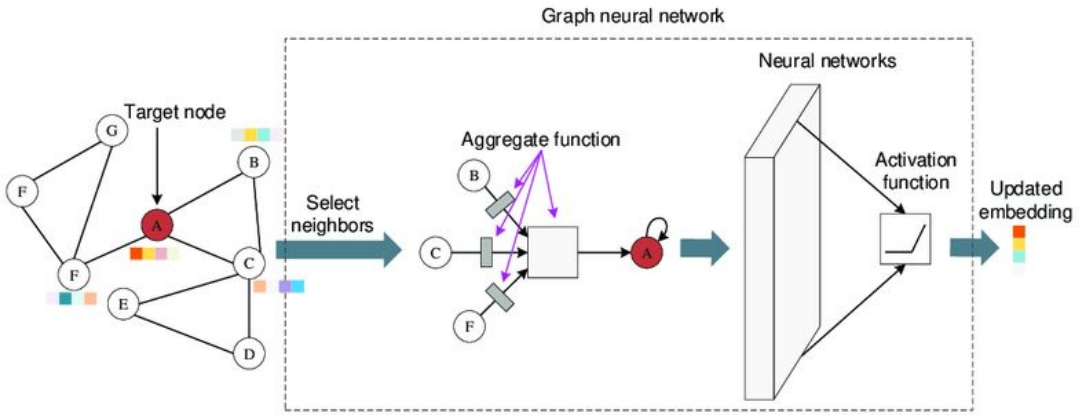


Every Ware Lab

Graph Neural Networks

Graph Neural Networks (GNNs)

- GNNs represent a framework for defining deep neural networks on graph data
- ***Idea***: for each node, learn an embedding representation
- This representation depends:
  - on the structure of the graph
  - on any feature information we might have
- The obtained nodes embedding representation are used to solve the tasks mentioned before



Every Ware Lab

Message Passing

- The underlying mechanism of GNNs is called *Message Passing*
- It allows updating the embedding of a node by aggregating the representations of its neighbors
- It is based on two functions:
  - ***AGGREGATE:*** it determines how to combine the information from the neighbors
  - ***UPDATE:*** it determines how to update the representation in the target node

Every Ware Lab

Message Passing: The Formula

Given a node *u,* let $\mathbf{h}_u^{(k)}$ be its representation at the k-th iteration of Message Passing. Let N(*u*) be the neighbors of *u*.

UPDATE and AGGREGATE are arbitrarily differentiable functions (e.g., MLPs).

$\mathbf{m}_{\mathcal{N}(u)}^{(k)}$ is the message aggregated from the neighbors

Aggregation works on a *set* to deal with permutation invariance.

$$\mathbf{h}_u^{(k+1)} = \text{UPDATE}^{(k)}\left(\mathbf{h}_u^{(k)}, \text{AGGREGATE}^{(k)}(\{\mathbf{h}_v^{(k)}, \forall v \in \mathcal{N}(u)\})\right)$$

$$= \text{UPDATE}^{(k)}\left(\mathbf{h}_u^{(k)}, \mathbf{m}_{\mathcal{N}(u)}^{(k)}\right),$$

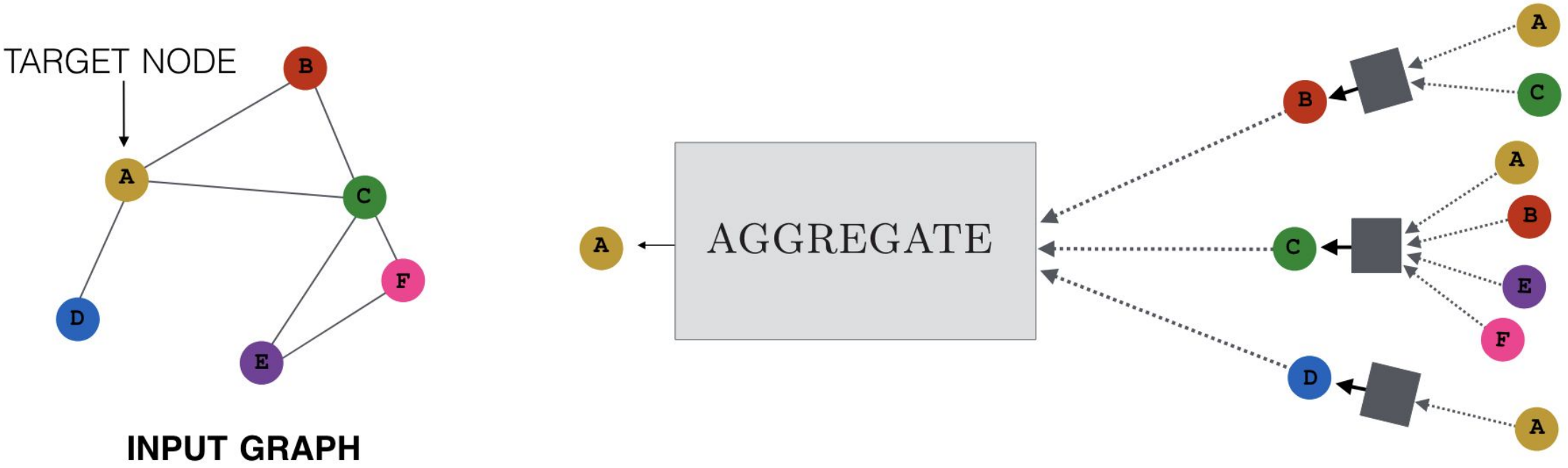
Every Ware Lab

How does it work? An intuition

- At time 0, each node is represented with its input features
- $\mathbf{h}^{(0)}_u = x_u \quad \forall u \in \mathbf{V}$
- At each iteration, each node aggregates information from direct neighbors
- As iterations progress, each node contains more and more infos from distant nodes
  - *e.g.,* at k=1 infos from 1-hop neighbors, at k=2 infos from 2-hops neighbors, etc
- At the end (e.g., after K iterations), each node is associated with an embedding encoding features of the K distant neighbors!
  - this is somehow analogous to CNN layers. Instead of spatial-defined patches, the GNN aggregates nodes from local neighbors!

Every Ware Lab

Message Passing visualized



Every Ware Lab

Naive Message Passing

Two learnable weights for each node.

Using a non-linear activation function (e.g., ReLU) on a linear combination.

Aggregation is simply the sum of neighbors representations.

$$\mathbf{h}_u^{(k)} = \sigma\left(\mathbf{W}_{\text{self}}^{(k)}\mathbf{h}_u^{(k-1)} + \mathbf{W}_{\text{neigh}}^{(k)} \sum_{v \in \mathcal{N}(u)} \mathbf{h}_v^{(k-1)} + \mathbf{b}^{(k)}\right)$$

In the following slides, the bias term will be omitted for simplicity.

Every Ware Lab

Naive Message Passing: AGGREGATE and UPDATE

$$\text{AGGREGATE}^{(k)}(\{\mathbf{h}_v^{(k)}, \forall v \in \mathcal{N}(u)\}) = \mathbf{m}_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \mathbf{h}_v$$

$$\text{UPDATE}(\mathbf{h}_u, \mathbf{m}_{\mathcal{N}(u)}) = \sigma\left(\mathbf{W}_{\text{self}}\mathbf{h}_u + \mathbf{W}_{\text{neigh}}\mathbf{m}_{\mathcal{N}(u)}\right)$$

Every Ware Lab

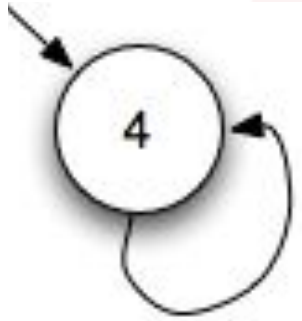
Message Passing with Self-Loops

Adding self-loops to each node for simplification, so to remove the UPDATE step!

$$\mathbf{h}_u^{(k)} = \text{AGGREGATE}(\{\mathbf{h}_v^{(k-1)}, \forall v \in \mathcal{N}(u) \cup \{u\}\})$$

Pros: it simplifies message passing, often alleviating overfitting. Only the weight for aggregation should be learned by the GNN.

Cons: it limits the expressivity of GNNs, since it's not possible to differentiate infos from neighbors with infos from the node itself!



Every Ware Lab

Normalization

- In the previous examples, aggregation was quite simple and based on the sum neighbors embeddings
- However, this may be unstable!
  - e.g. if a node *u* has 100x the number of neighbors than a node *u',* the sum may lead to drastic differences in magnitude!
- It is possible to perform normalization, or *symmetric* normalization to mitigate this problem!

$$\mathbf{m}_{\mathcal{N}(u)} = \frac{\sum_{v \in \mathcal{N}(u)} \mathbf{h}_v}{|\mathcal{N}(u)|}$$

Normalization

$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \frac{\mathbf{h}_v}{\sqrt{|\mathcal{N}(u)||\mathcal{N}(v)|}}$$

Symmetric Normalization

Every Ware Lab

Graph Convolutional Networks (GCNs)

Graph Convolutional Networks (GCNs)

Basically, GCNs combine self-loops and symmetric normalization.

$$\mathbf{h}_u^{(k)} = \sigma\left(\mathbf{W}^{(k)} \sum_{v \in \mathcal{N}(u) \cup \{u\}} \frac{\mathbf{h}_v}{\sqrt{|\mathcal{N}(u)||\mathcal{N}(v)|}}\right)$$

Convolution!

Every Ware Lab

Set Pooling

- Instead of summing, it is possible to do something more advanced for the **aggregation**
  - i.e., mapping a *set* of embeddings into a single embedding
- Known results in ML show that a permutation-invariant function mapping a set of embeddings to a single embedding can be approximated using the following formula (called *universal set function approximator*):

$$\mathbf{m}_{\mathcal{N}(u)} = \mathrm{MLP}_\theta \left(\sum_{v \in N(u)} \mathrm{MLP}_\phi(\mathbf{h}_v) \right)$$

- Here, MLP is an arbitrary deep multi-layer perceptron in charge of generating a latent representation of the input

Every Ware Lab

Janossy Pooling

- Alternative to Set Pooling, proved to be more powerful
- It leverages permutation-sensitive functions
- Goal: averaging over ***many*** possible permutations
- Let $\pi_i \in \mathbf{\Pi}$ be a permutation taking the unordered set of neighbor embeddings and placing these embeddings in a specific sequence based on some arbitrary ordering.

$$\mathbf{m}_{\mathcal{N}(u)} = \mathrm{MLP}_\theta \left(\frac{1}{|\Pi|} \sum_{\pi \in \Pi} \rho_\phi \left(\mathbf{h}_{v_1}, \mathbf{h}_{v_2}, \ldots, \mathbf{h}_{v_{|\mathcal{N}(u)|}} \right)_{\pi_i} \right)$$

ρ_ϕ is a permutation-sensitive function extracting embeddings (e.g., LSTM).

Since trying all the possible permutations is intractable, these are usually sampled.

Every Ware Lab

Graph-Level Embedding: *Graph Pooling*

- Message passing leads to a set of node embeddings
- What if we want to obtain a global latent representation of the graph?
- ***Goal:*** given nodes embeddings, $\forall z_u \in \mathbf{V}$, how to learn a representation z_G for the entire graph G?
- *Learning over sets:*

$$\mathbf{z}_{\mathcal{G}} = \frac{\sum_{v \in \mathcal{V}} \mathbf{z}_u}{f_n(|\mathcal{V}|)}$$

normalization function

Note that this may be further improved by using an attention mechanism, in charge of learning a weight for each z_u and performing a weighted sum!

Every Ware Lab

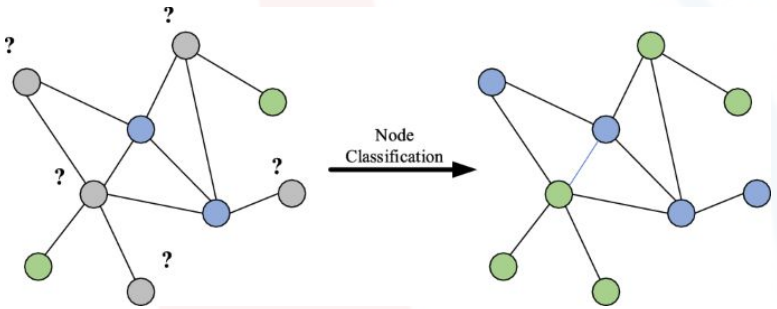
Let’s go back to our tasks!

Every Ware Lab

Node Classification

- Performing classification on each node based on the embeddings obtained after message passing
- In *transductive* scenarios, message passing can also leverage unlabeled nodes (semi-supervised)
- It is however crucial to ignore test nodes (also called inductive nodes) during training!
  - these nodes should not impact neither message passing and the training phase
- A possible loss function:

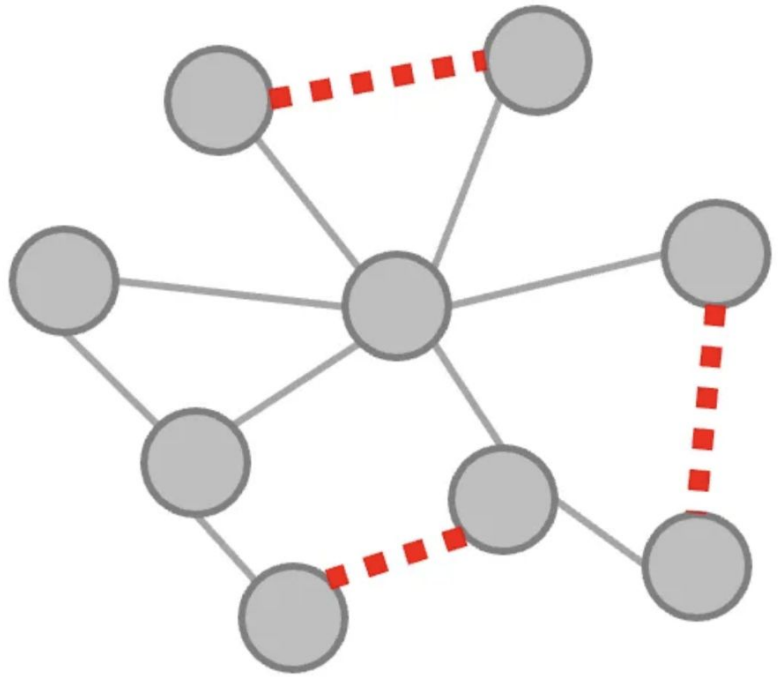
$$\mathcal{L} = \sum_{u \in \mathcal{V}_{\text{train}}} -\log(\text{softmax}(\mathbf{z}_u, \mathbf{y}_u)).$$



Every Ware Lab

Link Prediction

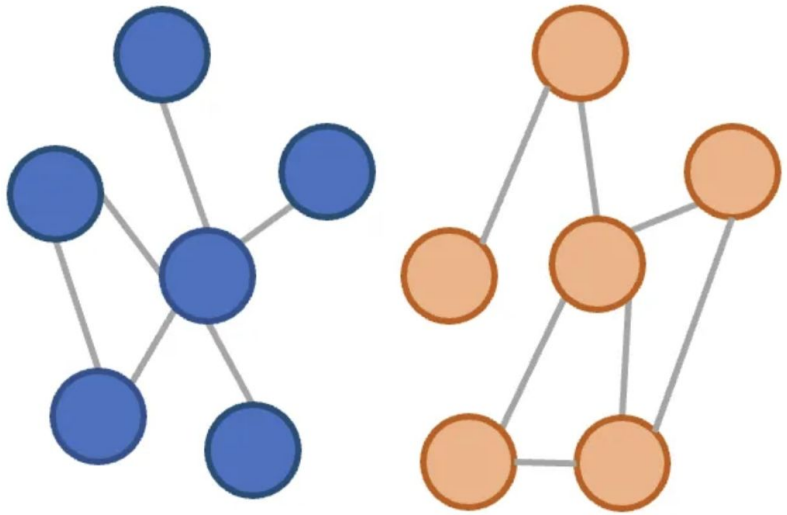
- Link prediction is usually framed as a metric learning problem:
  - minimizing the distance between the embeddings of nodes collected by a link
  - maximizing the distance between the embeddings of nodes not collected by a link
- Hence, a link is predicted between two nodes if their embeddings are similar (e.g., based on a threshold!)



Every Ware Lab

Graph Classification/Regression/Clustering

- This is the “simplest” case!
- It is sufficient to obtain the graph embedding z_G for each graph in the dataset, after graph pooling
- Then, it is possible to apply standard DL approaches
  - i.e., each Z_G is a data point



Every Ware Lab

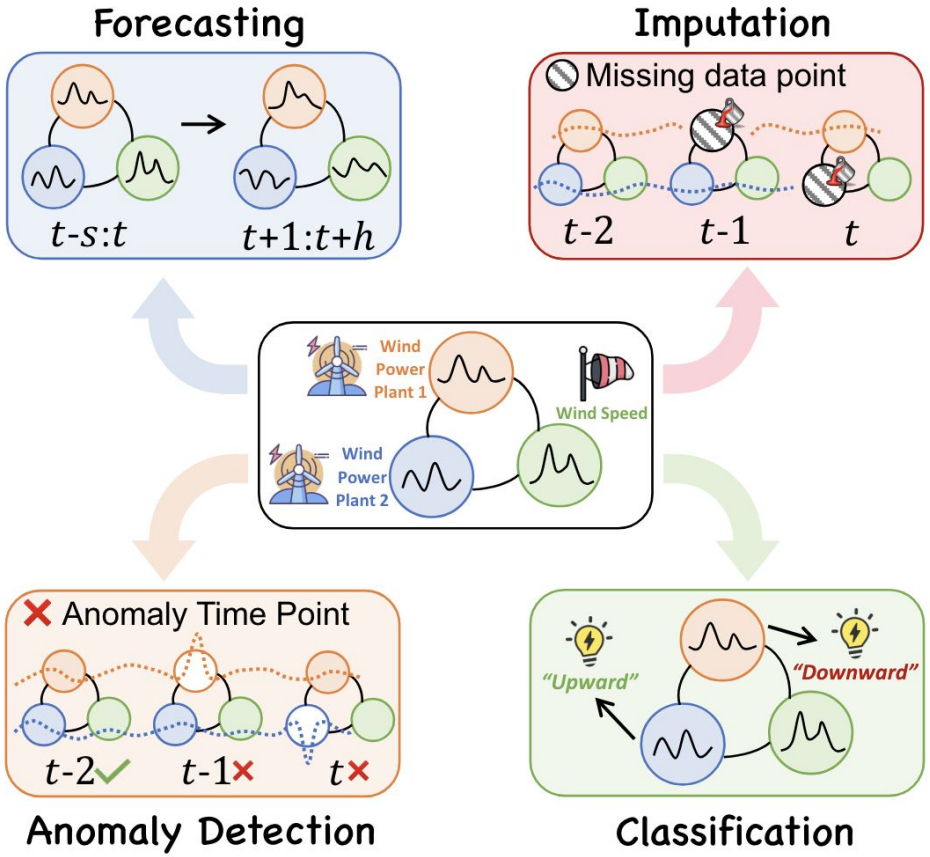
GNNs in Ambient Intelligence Scenarios

GNN in Ambient Intelligence

- There are several benefits of applying a GNN to model sensor data, besides what is provided by CNN and RNN
- CNNs and RNNs can somehow be viewed as GNNs
  - i.e., fixed-size grid graphs for CNNs and line graphs for RNNs
- Typical GNNs deal with complex graphs
  - no fixed form
  - variable size of unordered nodes
  - variable amount of neighbors for each node
- Advantages of GNNs:
  - effectively encode complex relationships and interdependencies among devices in IoT sensing systems, and among data instances over time!

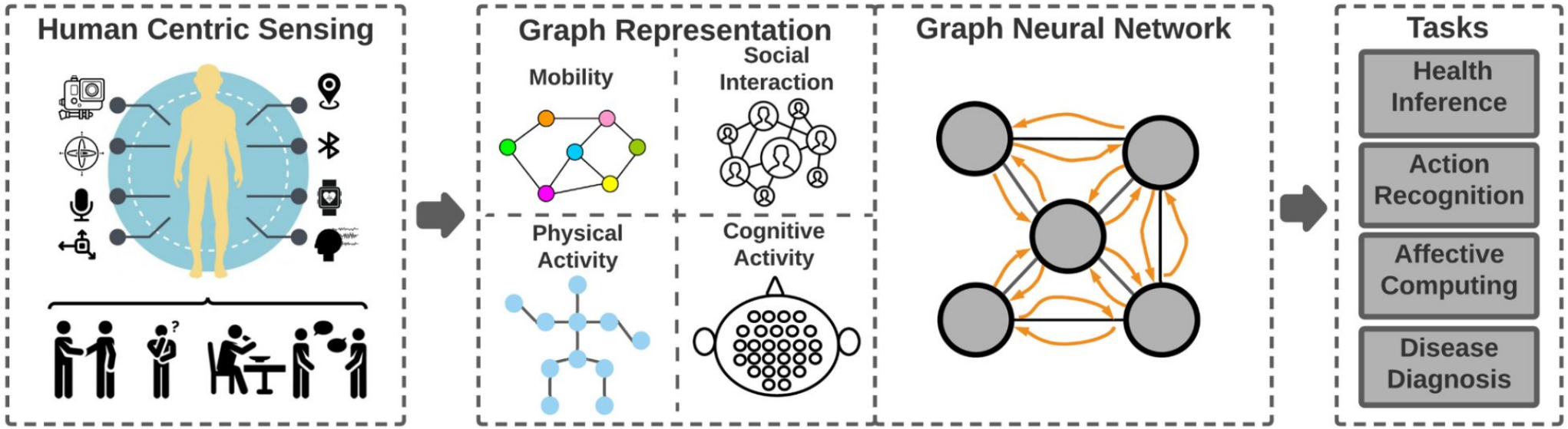
Every Ware Lab

Time series GNN tasks



Every Ware Lab

GNNs in Human Centric Sensing



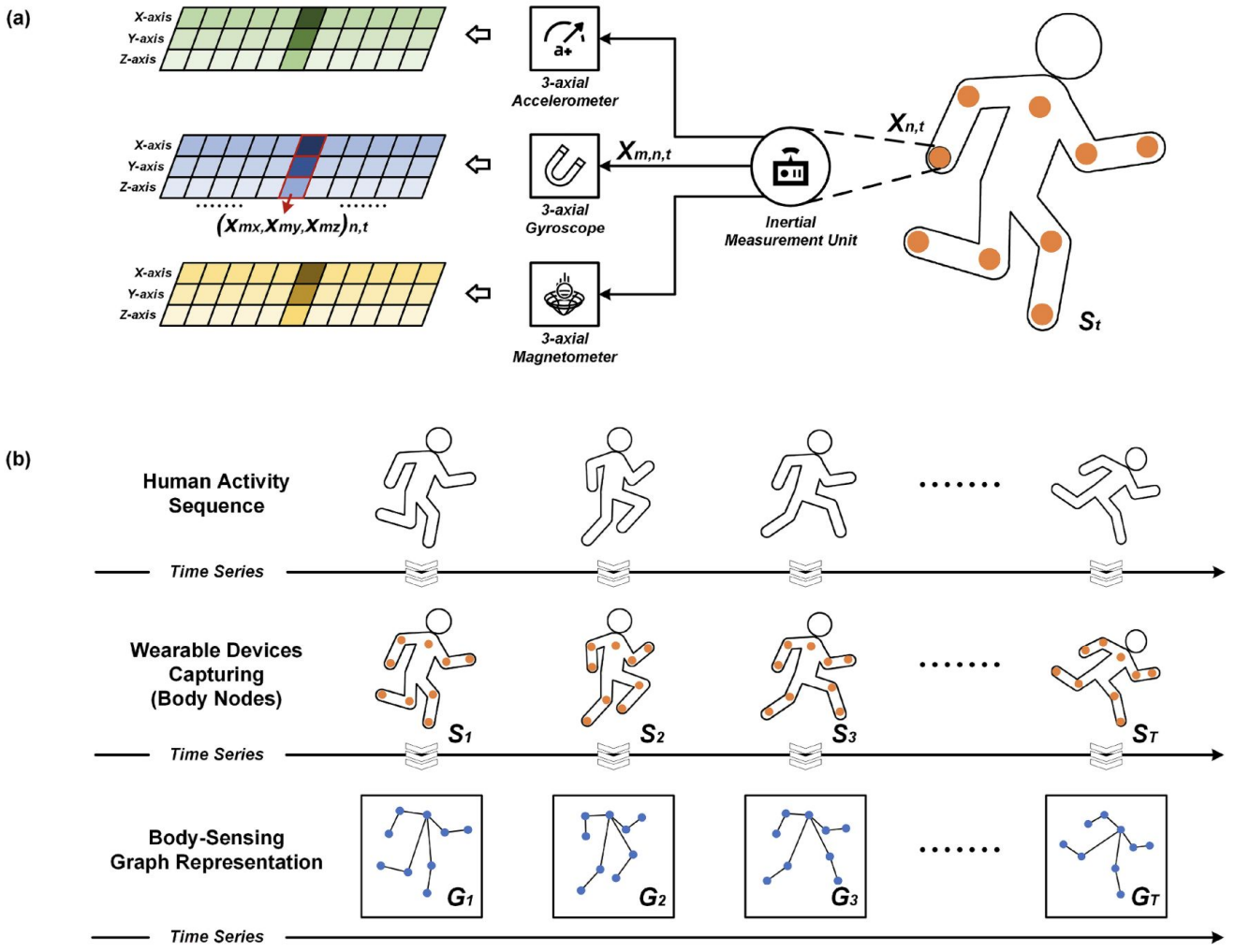
Every Ware Lab

Graph construction in Time Series

- How to construct a graph from a multivariate time series?
- Each node may represent a sensor (or a device with multiple sensors)
  - node features: a time window of sensor data
- How to set the edges between nodes?
  - **Heuristic-based graphs:**
    - using some heuristics like spatial proximity between sensors, pairwise similarity between sensors based on training data, etcetera
  - **Learning-based graphs:**
    - edges are learned in a end-to-end fashion. Initially, there is an edge between each pair of nodes
    - attention-based mechanisms can be used to derive the most important edges
    - more costly than heuristic-based, but it may lead to complex and informative graph structures

Every Ware Lab

Body-sensing Graph Representation for wearable-based HAR



Zou, H., Chen, Z., Zhang, J., Wang, L., Zhang, F., Li, J., & Pan, Y. (2024). GT-WHAR: A Generic Graph-Based Temporal Framework for Wearable Human Activity Recognition With Multiple Sensors. *IEEE Transactions on Emerging Topics in Computational Intelligence*.

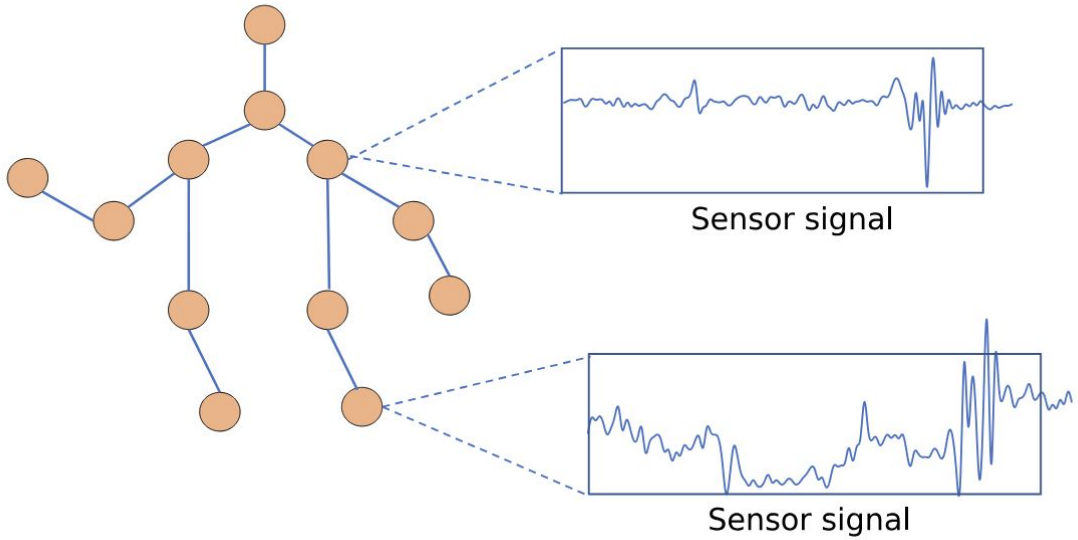
Every Ware Lab

Leveraging spatial relationships

Each node represents a device (possibly with multiple sensors) positioned in a body position.

Feature node: a time window of sensor data collected in that position.

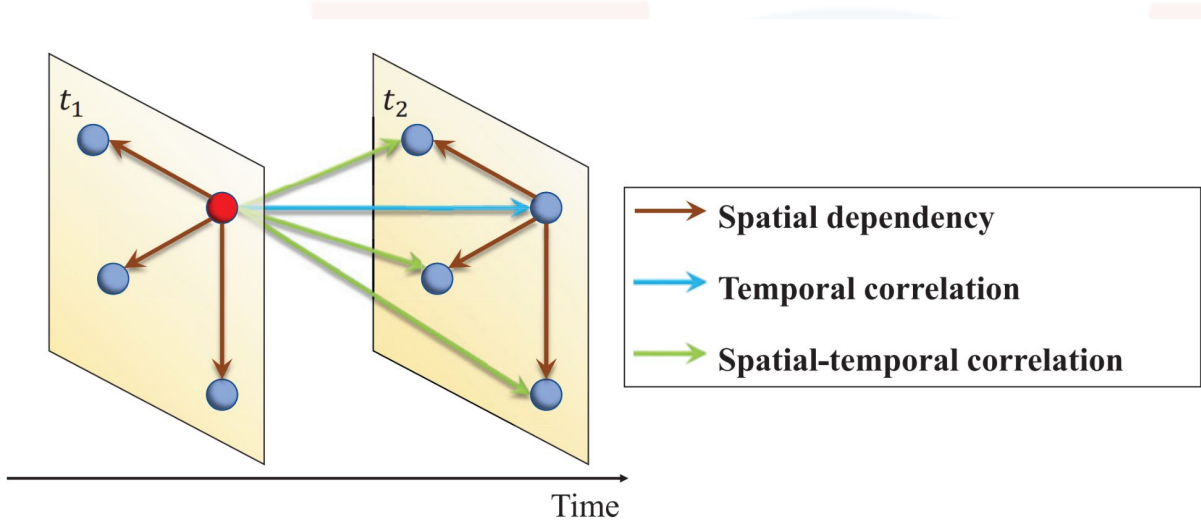
Edges directly connect adjacent body parts to capture dependencies between close devices!



Every Ware Lab

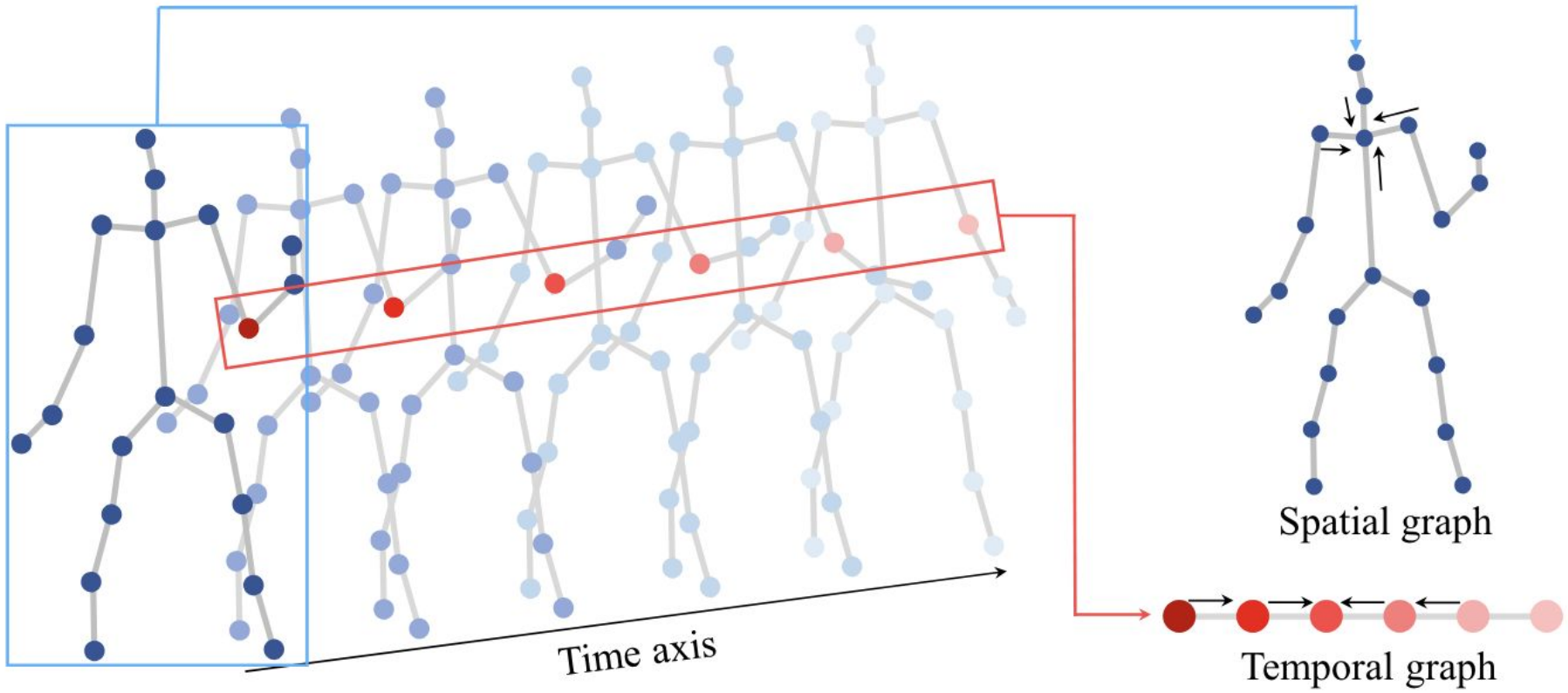
SpatioTemporal Graphs

- Why not encoding both space and time in the graph?
- ***Idea:*** Multi-relational graph, where different layers represent different time instants
  - the challenge is to define what an “instant” is (e.g., one second of sensor data data)
- ***Spatial dependency:*** Nodes connected by an edge at the same time instant
- ***Temporal correlation:*** The same node connected at different time instants
- **Spatial-temporal correlation:** Different nodes connected in different time instants



Every Ware Lab

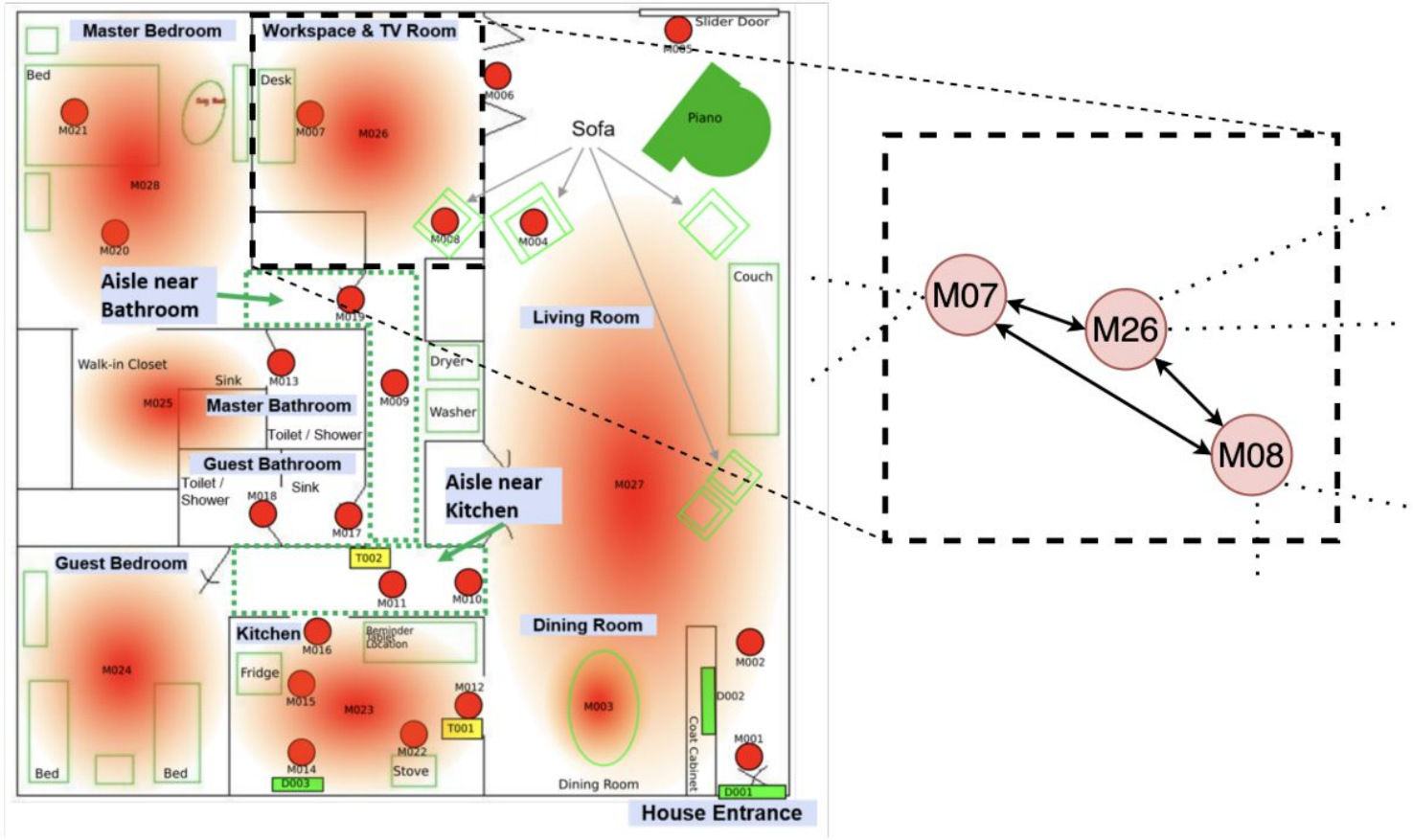
SpatioTemporal Graph with body-sensing graph representation



Li, M., Chen, S., Zhao, Y., Zhang, Y., Wang, Y., & Tian, Q. (2021). Multiscale spatio-temporal graph neural networks for 3d skeleton-based motion prediction. *IEEE Transactions on Image Processing*, *30*, 7760-7775.

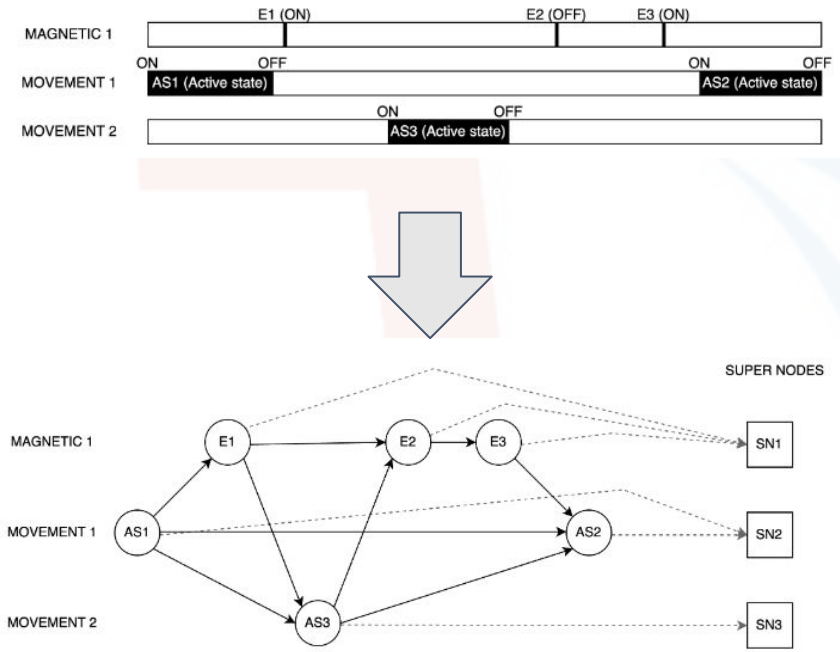
Every Ware Lab

GNNs for Smart-Home HAR



Graph construction in Smart-Homes (heuristic-based)

- In this approach, a window of sensor events is converted into a graph by encoding the temporal relationships between events
- Main idea:
  - each node (i.e., a sensor event) is connected with an edge to another node if they are temporally consecutive in the window

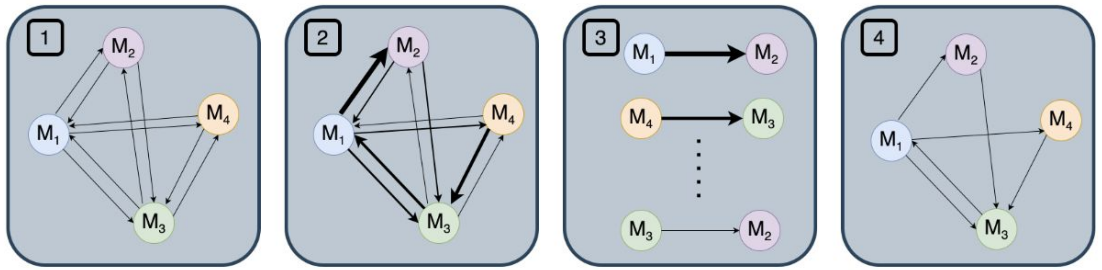


Fiori, M., Mor, D., Civitarese, G., & Bettini, C. (2025). GNN-XAR: A Graph Neural Network for Explainable Activity Recognition in Smart Homes. arXiv preprint arXiv:2502.17999.

Every Ware Lab

Graph construction in Smart-Homes (heuristic + learning)

- Start with a fully connected graph
  - each node is a sensor
- Compute pairwise similarity based on *sensor behavior*
  - e.g., frequency of activation, periodicity of triggering, range of measured values
- Keep only the edges with the highest similarity!
- Edges are then weighted with an attention mechanism
  - giving higher weights to couple of sensors that are most likely triggered together

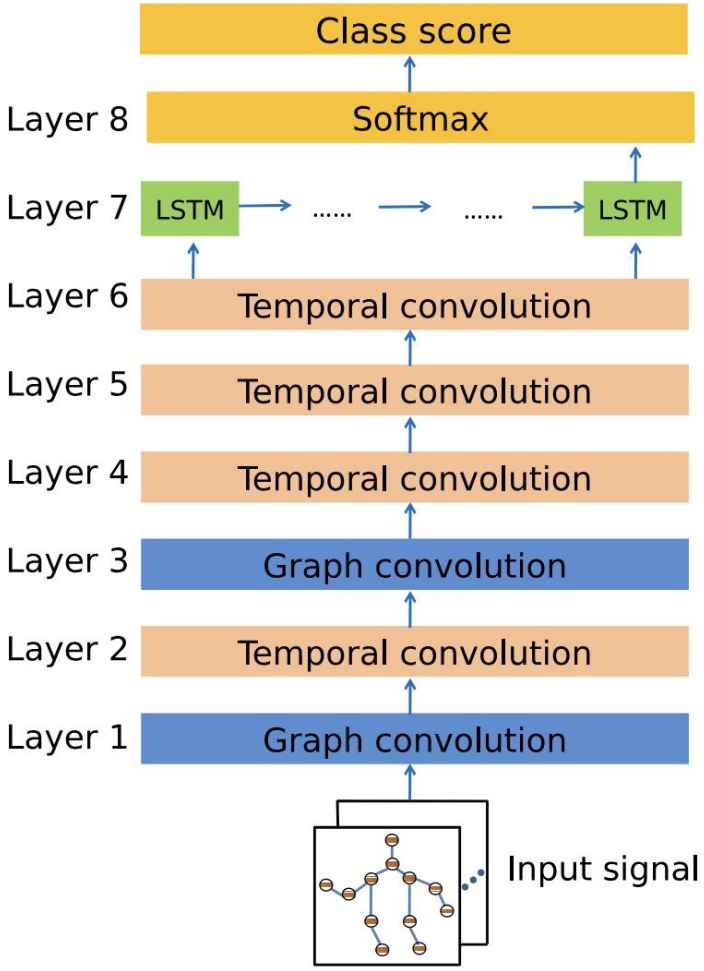


Plötz, T. (2023). Know Thy Neighbors: A Graph Based Approach for Effective Sensor-Based Human Activity Recognition in Smart Homes. *arXiv preprint arXiv:2311.09514*.

Every Ware Lab

A GNN for Time Series: GraphConvLSTM

- Graph Convolution is computed to capture the *spatial* properties in the graph
- Temporal Convolution is performed using a 1D-CNN layer applied individually on each node, so to capture *temporal* properties
- These two convolutions have the goal of learning node embeddings capturing spatio-temporal properties
- The overall time series is finally processed by LSTM to capture long-term temporal dependencies before classification

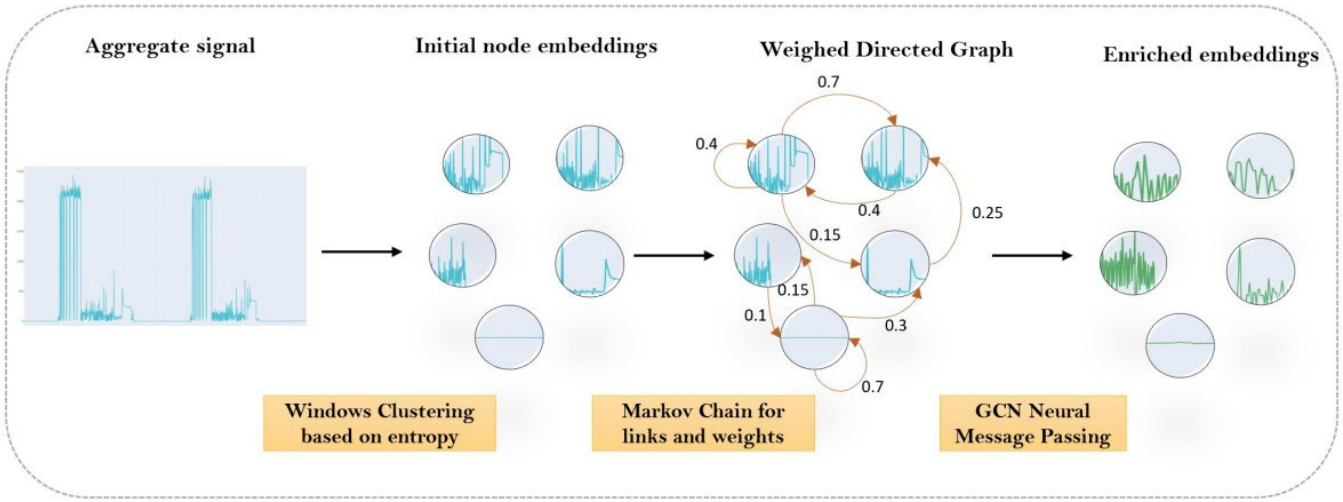


Every Ware Lab

[Han, J., He, Y., Liu, J., Zhang, Q., & Jing, X. (2019, December). GraphConvLSTM: Spatiotemporal learning for activity recognition with wearable sensors. In *2019 IEEE Global Communications Conference (GLOBECOM)* (pp. 1-6). IEEE.]

GNNs for Load Disaggregation

- The aggregated signal from the smart meter is *clustered* to identify different operational modes of appliances
  - each cluster is a node!
- Edges between nodes are weighted, where each weight is the probability of transitioning from one pattern to the other
- GCN is adopted for message passing, and then graph pooling is applied to obtain a single embedding of the aggregated signal
- The graph embedding is provided to a decoder that maps it to power consumption of individual appliances!

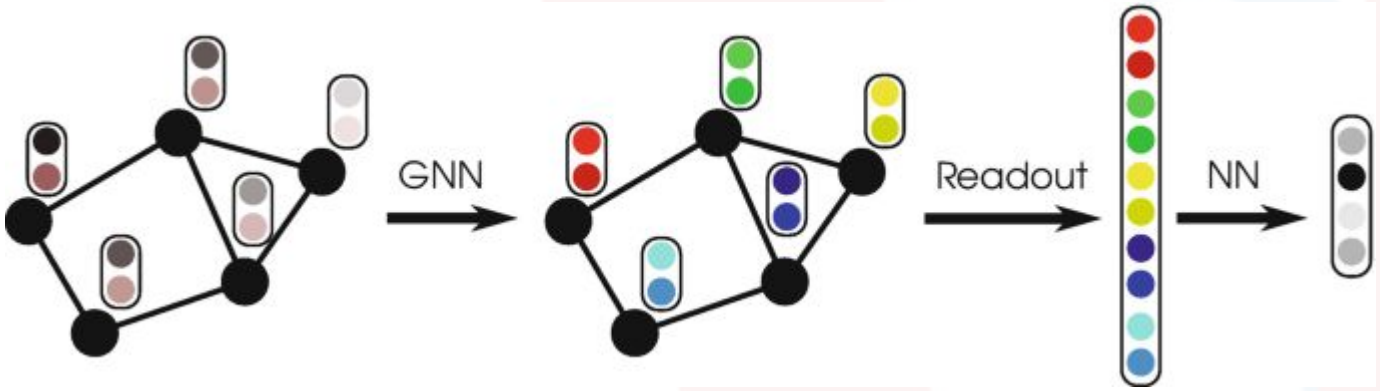


Athanasoulias, S., Sykiotis, S., Kaselimi, M., Protopapadakis, E., & Ipiotis, N. (2022, June). A first approach using graph neural networks on non-intrusive-load-monitoring. In *Proceedings of the 15th International Conference on PErvasive Technologies Related to Assistive Environments* (pp. 601-607).

Every Ware Lab

GNNs for Indoor Localization

- Each node represents a specific Access Point
  - the feature value is the measured RSSI
- A weighted edge between nodes is added if the two APs are spatially close
  - by looking at the map…
  - …or automatically, by mining the training set to find instances where the strength signal was high for both APs at the same time
- After message passing, graph pooling is performed to identify the position of the subject from the graph embedding



Lezama, F., Larroca, F., & Capdehourat, G. (2023). On the application of graph neural networks for indoor positioning systems. In *Machine Learning for Indoor Localization and Navigation* (pp. 239-256). Cham: Springer International Publishing.

Every Ware Lab

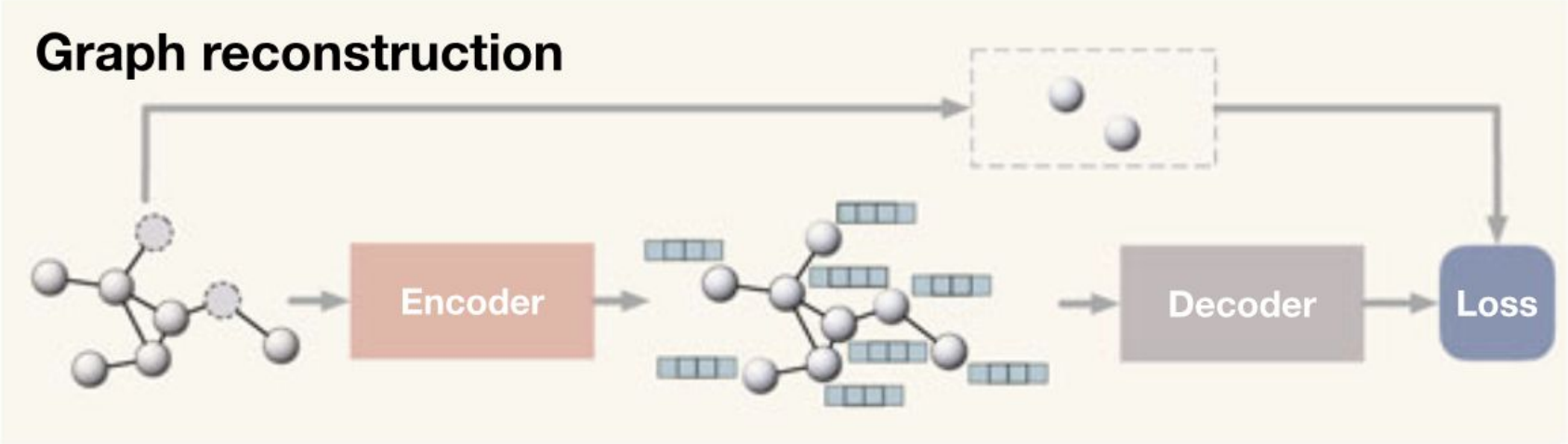
Advanced GNNs topics

Advanced GNN topics

- In order to connect this lecture with the previous ones we will answer the following questions:
  - can GNNs be used for self-supervised learning?
  - can we employ XAI methods for GNNs?

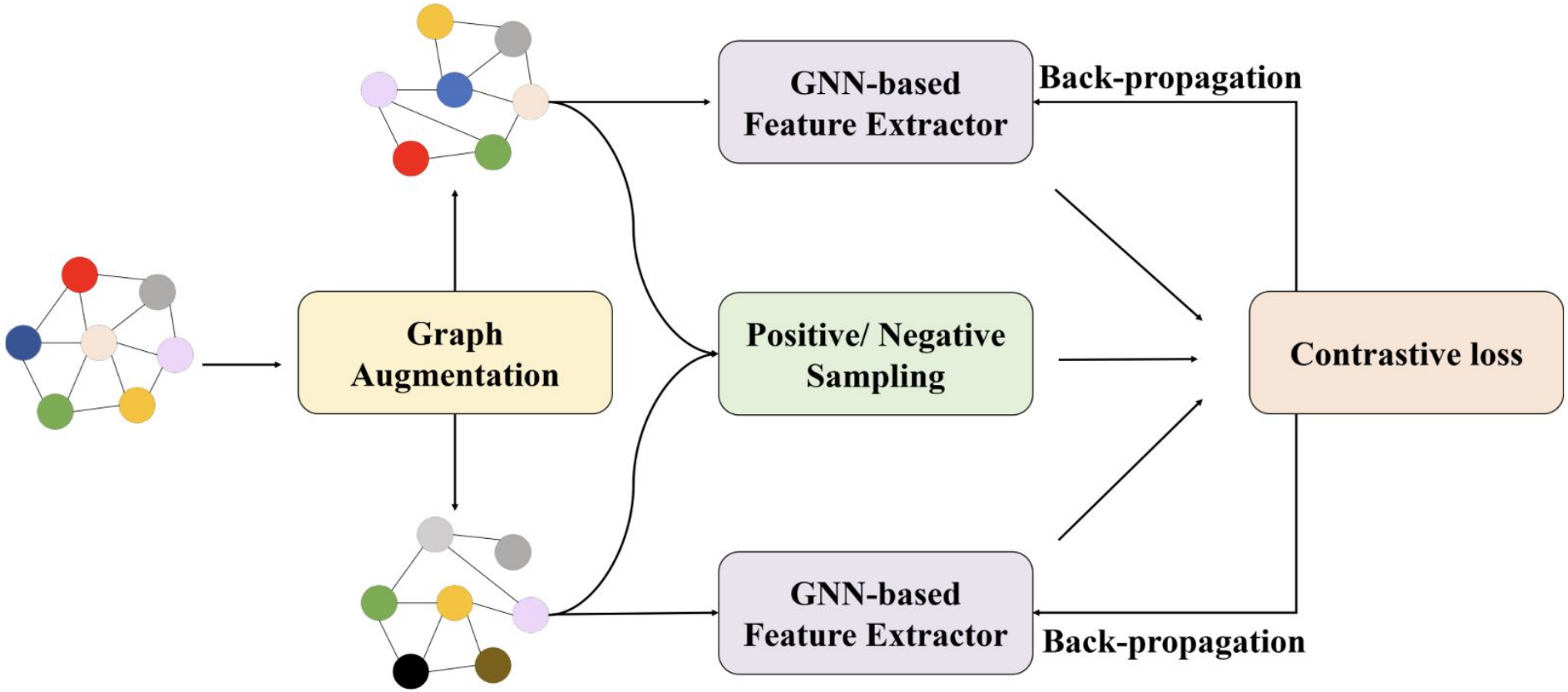
Every Ware Lab

Self-supervised Learning with GNNs: A Predictive Approach

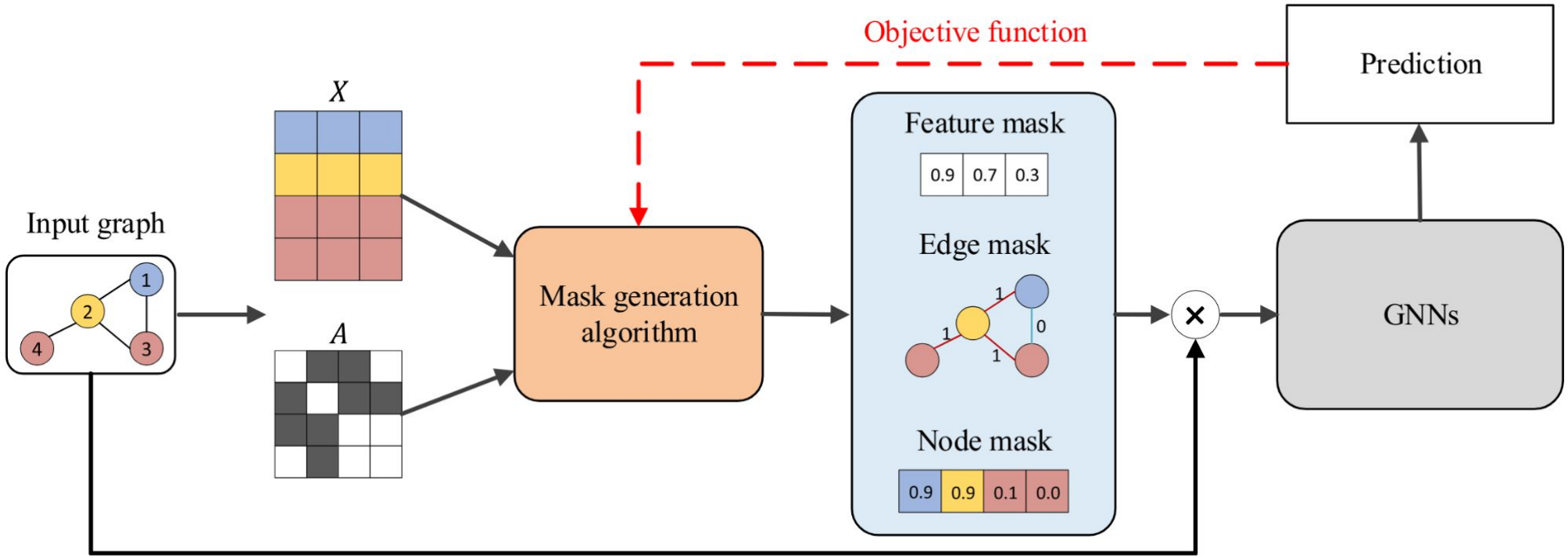


Every Ware Lab

Self-supervised Learning with GNNs: A Contrastive Approach



GNNs and Explainability: A perturbation-based Approach



Yuan, H., Yu, H., Gui, S., & Ji, S. (2022). Explainability in graph neural networks: A taxonomic survey. *IEEE transactions on pattern analysis and machine intelligence*, *45*(5), 5782-5799.

Every Ware Lab

References

[1] Hamilton, W. L. (2020). ***Graph representation learning.*** Morgan & Claypool Publishers.

[2] Dong, G., Tang, M., Wang, Z., Gao, J., Guo, S., Cai, L., ... & Boukhechba, M. (2023). ***Graph neural networks in IoT: A survey.*** ACM Transactions on Sensor Networks, 19(2), 1-50.

[3] Jin, M., Koh, H. Y., Wen, Q., Zambon, D., Alippi, C., Webb, G. I., ... & Pan, S. (2023). ***A survey on graph neural networks for time series: Forecasting, classification, imputation, and anomaly detection.*** arXiv preprint arXiv:2307.03759.

Every Ware Lab